

InterviewGen AI

Interview Test Report

Candidate Information

Name : Yaswanth Krishna
Email : vykrishna2006@gmail.com
Date : 5/7/2026
Status : Completed

Overall Score

1%

0 / 70 questions correct

Section-wise Results

AWS	2/50 (4%)
EC2	0/50 (0%)
Docker	0/50 (0%)
DSA	0/50 (0%)
DBMS	0/50 (0%)
DAA	0/50 (0%)
HR	0/50 (0%)

Detailed Question Analysis

Q1. [AWS] ' INCORRECT – 1/5

Question:

Explain the key components of an AWS VPC and how they enable network isolation and connectivity.

Your Answer:

controlling exactly how and where traffic flows, separating your infrastructure from other public or cloud environments

Ideal Answer:

A VPC (Virtual Private Cloud) is a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. Key components include:

- * **CIDR Block**: Defines the IP address range for the VPC.
- * **Subnets**: Partitions the VPC's CIDR block into smaller ranges. Can be public (with an Internet Gateway) or private (without direct internet access).
- * **Internet Gateway (IGW)**: Allows communication between instances in public subnets and the internet.
- * **NAT Gateway/Instance**: Allows instances in private subnets to initiate outbound connections to the internet while preventing inbound connections.
- * **Route Tables**: Determine where network traffic from a subnet or gateway is directed.
- * **Security Groups**: Act as virtual firewalls for instances to control inbound/outbound traffic at the instance level.
- * **Network ACLs (NACLs)**: Act as stateless firewalls for subnets to control inbound/outbound traffic at the subnet level.

AI Feedback:

The candidate's answer correctly identifies the core purpose and benefits of an AWS VPC, specifically highlighting network isolation and granular control over traffic flow. This demonstrates a high-level understanding of *why* one would use a VPC. However, the question explicitly asks to 'explain the key components' of a VPC. The candidate's response does not list or describe any specific components (e.g., Subnets, Internet Gateways, Route Tables, Security Groups, NACLs, etc.). Therefore, while the statements made are accurate, the answer largely misses the main thrust of the question.

Q2. [AWS] ' INCORRECT – 1/5

Question:

Describe different S3 storage classes and when you would choose one over another.

Your Answer:

Amazon Web Services (AWS) provides multiple Amazon S3 Storage Classes tailored for specific access patterns, performance needs, and cost requirements

Ideal Answer:

Amazon S3 offers various storage classes optimized for different access patterns and cost requirements:

- * **S3 Standard**: Default, highly durable, available, and performant storage for frequently accessed data. Good for general-purpose storage, dynamic websites, content distribution.
- * **S3 Intelligent-Tiering**: Automatically moves data between two access tiers (frequent and infrequent) based on access patterns, optimizing costs without performance impact. Ideal for data with unknown or changing access patterns.
- * **S3 Standard-IA (Infrequent Access)**: For long-lived, less frequently accessed data that requires rapid access when needed. Lower storage cost than Standard but higher retrieval cost. Good for backups, disaster recovery, analytics data.
- * **S3 One Zone-IA**: Similar to Standard-IA but stores data in a single Availability Zone, making it cheaper but less resilient to AZ loss. Suitable for reproducible data or secondary backups.
- * **S3 Glacier Instant Retrieval**: For long-lived data that is rarely accessed but requires millisecond retrieval when needed. Cheaper than IA classes, higher retrieval cost.
- * **S3 Glacier Flexible Retrieval**: For archiving data that can be retrieved in minutes to hours. Very low cost. Suitable for long-term archives.
- * **S3 Glacier Deep Archive**: Lowest-cost storage for long-term archives, with retrieval times of hours. Compliance archives, highly cost-sensitive long-term backups.

AI Feedback:

This is a good introductory sentence that correctly sets the stage for discussing S3 storage classes, highlighting their purpose in addressing different access patterns, performance, and cost requirements. However, it does not yet describe the different storage classes themselves or when to choose one over another, which is the core of the question.

Q3. [AWS] ' INCORRECT – 0/5

Question:

How would you design a highly available and scalable web application on AWS using EC2?

Your Answer:

hi

Ideal Answer:

To design a highly available and scalable web application on AWS using EC2, I would incorporate the following:

- * **Multi-AZ Deployment**: Deploy EC2 instances across multiple Availability Zones (AZs) to protect against single AZ failures.
- * **Auto Scaling Groups (ASG)**: Use ASGs to automatically adjust the number of EC2 instances based on demand (e.g., CPU utilization, network I/O, custom metrics). This ensures scalability and helps maintain availability by replacing unhealthy instances.
- * **Elastic Load Balancer (ELB)**: Place an ELB (e.g., Application Load Balancer for HTTP/S traffic) in front of the ASG to distribute incoming traffic across healthy instances in different AZs. The ELB also handles SSL termination and health checks.
- * **Stateless Application Design**: Ensure application servers are stateless, pushing session management to external services like ElastiCache (Redis) or DynamoDB. This allows instances to be added or removed easily without data loss.
- * **Managed Database Services**: Use Amazon RDS (Multi-AZ deployment for HA) or DynamoDB for database needs, offloading database management and ensuring data availability.
- * **Centralized Logging & Monitoring**: Use CloudWatch, CloudTrail, and perhaps a service like Kinesis Firehose + S3/Elasticsearch for centralized logging and monitoring to quickly identify and troubleshoot issues.

AI Feedback:

No answer provided.

Q4. [AWS] ' INCORRECT – 0/5

Question:

When would you choose Amazon RDS over Amazon DynamoDB, and vice-versa? Provide examples.

Your Answer:

hi

Ideal Answer:

- * **Choose Amazon RDS (Relational Database Service) when:**
 - * You need a traditional relational database (SQL) with ACID compliance.
 - * Your data has a well-defined, rigid schema and complex joins are frequent.
 - * You need strong data consistency and transaction support.
 - * Examples: E-commerce platforms with complex product catalogs and order processing, financial applications requiring strict data integrity, content management systems.
- * **Choose Amazon DynamoDB (NoSQL Database) when:**
 - * You need a highly scalable, low-latency, non-relational database.
 - * Your data model is flexible and schema-less, or semi-structured.
 - * You require massive scale for read/write operations (millions of requests per second) and predictable performance.
 - * You need automatic scaling, high availability, and built-in replication across multiple AZs.
 - * Examples: Gaming leaderboards, IoT device data streams, real-time bidding platforms, user profiles for large-scale applications.

AI Feedback:

No answer provided.

Q5. [AWS] ' INCORRECT – 0/5

Question:

Explain the benefits and use cases of AWS Lambda. What are some limitations?

Your Answer:

Not answered

Ideal Answer:

* **Benefits of AWS Lambda**:

- * **No Server Management**: You don't provision or manage servers; AWS handles all infrastructure.
- * **Automatic Scaling**: Scales automatically with demand, from a few requests per day to thousands per second.
- * **Cost-Effective**: Pay only for the compute time consumed, charged per 100ms. No idle costs.
- * **High Availability**: Built-in fault tolerance and availability.
- * **Event-Driven**: Easily triggered by events from over 200 AWS services (S3, DynamoDB, API Gateway, Kinesis, etc.).

* **Use Cases**:

- * **Data Processing**: Thumbnail generation on S3 uploads, ETL processes.
- * **Web Backends**: RESTful APIs via API Gateway, handling user requests.
- * **Real-time Stream Processing**: Processing data from Kinesis or DynamoDB Streams.
- * **Scheduled Tasks**: Running cron jobs.
- * **Chatbots/Voice Assistants**: Processing requests from Alexa, Slack.

* **Limitations**:

- * **Execution Duration**: Default maximum execution time is 15 minutes.
- * **Memory/Disk Limits**: Restricted memory (up to 10GB) and ephemeral disk space (/tmp up to 10GB).
- * **Cold Starts**: Initial invocations of infrequently used functions might experience a slight delay due to container initialization.
- * **Concurrency Limits**: Default regional concurrency limits can be hit if not managed.
- * **Debugging Complexity**: Debugging distributed serverless architectures can be more challenging.

AI Feedback:

No answer provided.

Q6. [AWS] ' INCORRECT – 0/5

Question:

What is AWS IAM, and how does it help secure your AWS resources?

Your Answer:

Not answered

Ideal Answer:

AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. It allows you to manage users, groups, roles, and their permissions.

* **Key Concepts**:

- * **Users**: Entities representing a person or application that interacts with AWS.
- * **Groups**: Collections of IAM users, simplifying permission management for multiple users.
- * **Roles**: IAM entities that define a set of permissions for making AWS service requests. They are typically assumed by AWS services (e.g., EC2 instance role) or trusted external identities (e.g., cross-account access).

- * **Policies**: Documents that define permissions (what actions are allowed or denied on which resources). Policies can be attached to users, groups, or roles.

* **How it helps secure resources**:

- * **Principle of Least Privilege**: IAM enables you to grant only the necessary permissions required to perform a task, minimizing potential security risks.
- * **Granular Access Control**: Policies allow very specific control over which resources can be accessed and what actions can be performed (e.g., "Allow S3:GetObject on bucket 'my-app-data' only").
- * **Identity Federation**: Allows existing enterprise identities (e.g., Active Directory) to access AWS resources without recreating users in IAM.
- * **MFA (Multi-Factor Authentication)**: Can be enforced for IAM users to add an extra layer of security.
- * **Auditability**: Integrates with AWS CloudTrail to log all API calls, including who made them and from where, aiding in security audits.

AI Feedback:

No answer provided.

Q7. [AWS] ' INCORRECT – 0/5

Question:

Differentiate between AWS CloudWatch and AWS CloudTrail. When would you use each?

Your Answer:

Not answered

Ideal Answer:

* **AWS CloudWatch**:

- * **What it is**: A monitoring and observability service that provides data and actionable insights to monitor your applications, respond to system-wide performance changes, and optimize resource utilization.

- * **What it does**: Collects and tracks metrics (e.g., CPU utilization, network I/O, custom application metrics), creates dashboards, sets alarms based on metric thresholds, and reacts to events.

- * **When to use**: For operational monitoring, performance analysis, health checks, capacity planning, and triggering actions based on application or infrastructure performance. E.g., setting an alarm if EC2 CPU utilization exceeds 80% for 5 minutes.

* **AWS CloudTrail**:

- * **What it is**: A service that enables governance, compliance, operational auditing, and risk auditing of your AWS account.

- * **What it does**: Logs API calls made by users, roles, or AWS services in your account. These logs include who made the call, when, from where, and what resources were affected.

- * **When to use**: For security analysis, tracking changes to AWS resources, troubleshooting operational issues (e.g., identifying who deleted a resource), and meeting compliance requirements. E.g., finding out who stopped an EC2 instance or changed an S3 bucket policy.

AI Feedback:

No answer provided.

Q8. [AWS] ' INCORRECT – 0/5

Question:

Explain different disaster recovery strategies on AWS and factors influencing your choice.

Your Answer:

Not answered

Ideal Answer:

AWS offers several disaster recovery (DR) strategies, each with different RTO (Recovery Time Objective) and RPO (Recovery Point Objective) and cost implications:

- * **Backup and Restore**: Simplest and cheapest. Data is backed up to S3 (e.g., using EBS snapshots, RDS snapshots). If disaster strikes, restore from backups to a new environment. High RTO/RPO.

- * **Use case**: Non-critical applications, dev/test environments.

- * **Pilot Light**: Core infrastructure is always running in the DR region (e.g., database, minimal EC2 instances), ready to scale up. Data is continuously replicated. Moderate RTO/RPO and cost.

- * **Use case**: Business-critical applications that can tolerate some downtime.

- * **Warm Standby**: A scaled-down but fully functional version of the application is running in the DR region. Data is continuously replicated. Lower RTO/RPO and higher cost than pilot light.

- * **Use case**: Critical applications requiring minimal downtime.

- * **Multi-Site (Hot/Active-Active)**: Full production environment runs in multiple regions simultaneously, actively serving traffic. Data is replicated in real-time. Lowest RTO/RPO but highest cost.

- * **Use case**: Mission-critical applications where any downtime or data loss is unacceptable.

* **Factors influencing choice**:

- * **RTO (Recovery Time Objective)**: Maximum acceptable downtime.

- * **RPO (Recovery Point Objective)**: Maximum acceptable data loss.

- * **Cost**: Each strategy has different infrastructure and data transfer costs.

- * **Complexity**: Implementation and management effort.

- * **Compliance Requirements**: Industry regulations might mandate specific RTO/RPO.

AI Feedback:

No answer provided.

Q9. [AWS] ' INCORRECT – 0/5

Question:

What are the main differences between AWS ECS and EKS? When would you choose one over the other?

Your Answer:

Not answered

Ideal Answer:

Both ECS and EKS are container orchestration services on AWS.

* **AWS ECS (Elastic Container Service)**:

* **What it is**: A highly scalable, high-performance container orchestration service that supports Docker containers. AWS manages the control plane.

* **Key Features**: Simpler to set up and operate as it's an AWS-native service. Deep integration with other AWS services. Can use EC2 instances or AWS Fargate (serverless compute for containers).

* **When to choose**: When you need a fully managed container orchestration solution that is tightly integrated with AWS, prefer a simpler operational model, or are already heavily invested in the AWS ecosystem. Good for teams that want to focus purely on applications, not Kubernetes complexities.

* **AWS EKS (Elastic Kubernetes Service)**:

* **What it is**: A managed Kubernetes service that makes it easy to deploy, manage, and scale containerized applications using Kubernetes on AWS. AWS manages the Kubernetes control plane.

* **Key Features**: Provides the full power and flexibility of open-source Kubernetes. Enables portability of applications across hybrid and multi-cloud environments. Requires more Kubernetes specific knowledge. Can use EC2 instances, Fargate, or Bottlerocket for worker nodes.

* **When to choose**: When you need Kubernetes' rich feature set, want to leverage the vibrant Kubernetes ecosystem, need multi-cloud or hybrid cloud portability, or your team already has strong Kubernetes expertise. Ideal for organizations standardizing on Kubernetes across different environments.

AI Feedback:

No answer provided.

Q10. [AWS] ' INCORRECT – 0/5

Question:

What are some strategies for optimizing costs in AWS?

Your Answer:

Not answered

Ideal Answer:

Cost optimization on AWS is a continuous process involving several strategies:

* **Right-sizing Instances**: Regularly analyze resource utilization (CPU, memory, network) using CloudWatch metrics to ensure EC2 instances, RDS databases, etc., are not over-provisioned. Downgrade to smaller instance types when appropriate.

* **Leveraging Reserved Instances (RIs) or Savings Plans**: For stable, predictable workloads, RIs and Savings Plans offer significant discounts (up to 72%) compared to On-Demand pricing for commitments of 1 or 3 years.

* **Utilizing Spot Instances**: For fault-tolerant, flexible, and interruptible workloads (e.g., batch processing, data analytics), Spot Instances can provide up to 90% savings compared to On-Demand prices.

* **Adopting Serverless Architectures**: Services like AWS Lambda, S3, DynamoDB, and Fargate are "pay-per-use" or "pay-per-request," eliminating idle costs and reducing operational overhead.

* **Optimizing S3 Storage**: Use appropriate S3 storage classes (Intelligent-Tiering, Standard-IA, Glacier) based on data access patterns, and implement lifecycle policies to transition or expire old data.

* **Eliminating Idle/Unused Resources**: Identify and terminate unused EC2 instances, EBS volumes, old snapshots, unattached Elastic IPs, and idle RDS instances.

* **Data Transfer Costs**: Minimize data transfer out of AWS regions or across AZs when possible, as egress costs can be significant. Use CloudFront for content delivery.

* **Monitoring and Budgeting**: Use AWS Cost Explorer, Budgets, and Cost & Usage Reports to track spending, identify trends, and set alerts for budget overruns.

* **Architectural Efficiency**: Design architectures for efficiency, leveraging services like SQS for decoupling components, which can reduce the need for constantly running compute resources.

AI Feedback:

No answer provided.

Q11. [EC2] ' INCORRECT – 0/5

Question:

Explain what Amazon EC2 is and describe the essential steps to launch a new EC2 instance.

Your Answer:

Not answered

Ideal Answer:

Amazon EC2 (Elastic Compute Cloud) provides resizable compute capacity in the cloud. It allows users to rent virtual servers (instances) on which to run their own computer applications. The essential steps to launch an instance include: choosing an Amazon Machine Image (AMI), selecting an instance type, configuring instance details (VPC, subnet, IAM role), adding storage (EBS volumes), configuring security groups, and reviewing/launching.

AI Feedback:

No answer provided.

Q12. [EC2] ' INCORRECT – 0/5

Question:

Differentiate between various EC2 instance families (e.g., General Purpose, Compute Optimized, Memory Optimized, Storage Optimized, Accelerated Computing) and provide a scenario where you would choose each.

Your Answer:

Not answered

Ideal Answer:

General Purpose (M, T) for balanced workloads like web servers. Compute Optimized (C) for compute-intensive tasks like HPC or video encoding. Memory Optimized (R, X, U) for large in-memory databases or analytics. Storage Optimized (I, D, H) for high I/O NoSQL databases or data warehousing. Accelerated Computing (P, G, F) for machine learning or graphics rendering using GPUs/FPGAs.

AI Feedback:

No answer provided.

Q13. [EC2] ' INCORRECT – 0/5

Question:

Explain the key differences between EC2 Security Groups and Network Access Control Lists (Network ACLs), and when you would use one over the other or both.

Your Answer:

Not answered

Ideal Answer:

Security Groups are stateful virtual firewalls at the instance level, allowing only 'allow' rules. Network ACLs are stateless virtual firewalls at the subnet level, allowing both 'allow' and 'deny' rules evaluated in order. Security Groups are mandatory for instance security; Network ACLs provide an optional, additional layer of subnet security, often used for more granular control or deny rules.

AI Feedback:

No answer provided.

Q14. [EC2] ' INCORRECT – 0/5

Question:

Describe the differences between Amazon EBS (Elastic Block Store) and EC2 Instance Store volumes. When would you choose one over the other?

Your Answer:

Not answered

Ideal Answer:

EBS volumes are persistent, network-attached storage; data remains even if the instance is stopped or terminated. They offer various performance types and can be snapshotted. Instance Store volumes are ephemeral storage physically attached to the host; data is lost when the instance is stopped, terminated, or fails, but they offer very high I/O performance. Choose EBS for persistent data, and Instance Store for temporary data, caches, or replicated data that can tolerate loss.

AI Feedback:

No answer provided.

Q15. [EC2] ' INCORRECT – 0/5

Question:

Explain how an Auto Scaling Group (ASG) works and list the key components required to configure one for an EC2 application.

Your Answer:

Not answered

Ideal Answer:

An ASG automatically adjusts the number of EC2 instances based on demand or schedules, ensuring desired capacity. Key components include: a Launch Template (or Launch Configuration) defining instance parameters, Min/Max/Desired capacity, VPC and subnets, Scaling Policies (e.g., target tracking, scheduled) often linked to CloudWatch alarms, and Health Checks. Integration with a Load Balancer's Target Group is common.

AI Feedback:

No answer provided.

Q16. [EC2] ' INCORRECT – 0/5

Question:

Compare and contrast On-Demand, Reserved, and Spot EC2 instances, highlighting their primary use cases and cost implications.

Your Answer:

Not answered

Ideal Answer:

On-Demand instances offer pay-as-you-go pricing, suitable for irregular or unpredictable workloads. Reserved Instances (RIs) provide significant discounts for a 1 or 3-year commitment, ideal for stable, continuous workloads. Spot Instances offer deep discounts (up to 90%) by bidding on unused capacity, but can be interrupted; best for fault-tolerant, flexible workloads like batch processing. On-Demand is highest cost, followed by RIs, then Spot is lowest.

AI Feedback:

No answer provided.

Q17. [EC2] ' INCORRECT – 0/5

Question:

How would you monitor the performance and health of your EC2 instances? What metrics are important, and what tools would you use?

Your Answer:

Not answered

Ideal Answer:

I would primarily use Amazon CloudWatch. Important metrics include CPU Utilization, Network In/Out, Disk Read/Write Ops/Bytes (default). For deeper insights like memory or disk space, I'd install the CloudWatch Agent for custom metrics. I would configure CloudWatch Alarms on critical metrics to trigger notifications (SNS) or automated actions, and use CloudWatch Dashboards for visual monitoring. CloudWatch Logs for log aggregation, and AWS Systems Manager for operational insights and management.

AI Feedback:

No answer provided.

Q18. [EC2] ' INCORRECT – 0/5

Question:

An EC2 instance that was previously running fine is now unreachable via SSH/RDP. What steps would you take to diagnose and resolve the issue?

Your Answer:

Not answered

Ideal Answer:

1. Check Instance Status Checks (System and Instance) in EC2 console. 2. Verify Security Group rules allow inbound SSH/RDP from your IP. 3. Check Network ACLs on the subnet allow relevant traffic. 4. Review System Log/Console Output in the EC2 console for boot issues. 5. Monitor CloudWatch metrics (CPU, memory, disk I/O) for resource exhaustion. 6. Use AWS Systems Manager Session Manager (if configured) for in-OS troubleshooting. 7. As a last resort, reboot or stop/start the instance. If data recovery is critical, detach the root EBS volume and attach it to another healthy instance for inspection.

AI Feedback:

No answer provided.

Q19. [EC2] ' INCORRECT – 0/5

Question:

What are EC2 Placement Groups, and when would you use each type (Cluster, Spread, Partition)?

Your Answer:

Not answered

Ideal Answer:

EC2 Placement Groups allow influencing instance placement for specific reliability, performance, or availability needs. Cluster Placement Groups co-locate instances within an AZ for low-latency networking (e.g., HPC). Spread Placement Groups distribute instances across distinct hardware racks within an AZ to reduce correlated failures (e.g., critical small-scale applications). Partition Placement Groups divide instances into logical partitions across distinct racks for large distributed systems (e.g., HDFS, Cassandra) to minimize correlated failures.

AI Feedback:

No answer provided.

Q20. [EC2] ' INCORRECT – 0/5

Question:

You need to deploy a fleet of EC2 instances with a specific OS configuration, pre-installed software, and custom scripts. How would you achieve this efficiently, and what considerations are there for sharing this configuration?

Your Answer:

Not answered

Ideal Answer:

To achieve this efficiently, I would create a 'Golden AMI'. This involves launching a base instance, configuring it with the desired OS settings, installing software, and adding custom scripts. Then, I would create an AMI from this configured instance. For deployment, this custom AMI would be used in Auto Scaling Groups or Launch Templates. Considerations for sharing include: sharing within the same AWS account, explicitly sharing with specific AWS account IDs, and understanding that AMIs are region-specific and must be copied to other regions. Public sharing is generally discouraged for security reasons. Regular updates and rebuilding of AMIs are crucial for security patches.

AI Feedback:

No answer provided.

Q21. [Docker] ' INCORRECT – 0/5

Question:

What is Docker, and what problem does it aim to solve in software development and deployment?

Your Answer:

Not answered

Ideal Answer:

Docker is a platform for developing, shipping, and running applications in containers. It solves the problem of 'it works on my machine' by packaging an application and its dependencies into a single, isolated unit called a container. This ensures consistency across different environments (development, testing, production) and simplifies deployment, scaling, and management of applications.

AI Feedback:

No answer provided.

Q22. [Docker] ' INCORRECT – 0/5

Question:

Explain the key differences between a Docker Image and a Docker Container.

Your Answer:

Not answered

Ideal Answer:

A Docker Image is a lightweight, standalone, executable package that includes everything needed to run a piece of software, including the code, a runtime, libraries, environment variables, and config files. It's a read-only template. A Docker Container is a runnable instance of an image. It's a live, running process that has its own isolated environment and file system. You can create, start, stop, move, or delete a container.

AI Feedback:

No answer provided.

Q23. [Docker] ' INCORRECT – 0/5

Question:

Describe the purpose of a Dockerfile. Name at least three common instructions you would find in one and explain their use.

Your Answer:

Not answered

Ideal Answer:

A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image. It serves as a blueprint for building Docker images programmatically. Common instructions include:

- `FROM`: Specifies the base image for subsequent instructions. (e.g., `FROM ubuntu:20.04`)
- `RUN`: Executes any commands in a new layer on top of the current image and commits the results. (e.g., `RUN apt-get update && apt-get install -y nginx`)
- `COPY` or `ADD`: Copies files or directories from the host to the container's filesystem. (`COPY ./app`)
- `EXPOSE`: Informs Docker that the container listens on the specified network ports at runtime. (e.g., `EXPOSE 80`)
- `CMD` or `ENTRYPOINT`: Specifies the command to execute when a container is launched from the image.

AI Feedback:

No answer provided.

Q24. [Docker] ' INCORRECT – 0/5

Question:

How do you persist data in Docker containers? Explain the concept of Docker volumes and why they are preferred over bind mounts in many scenarios.

Your Answer:

Not answered

Ideal Answer:

Data persistence in Docker is primarily achieved using Docker volumes. A volume is a designated filesystem managed by Docker that resides on the host system but outside the container's lifecycle. Unlike bind mounts, which directly map a host directory, volumes are fully managed by Docker and are typically preferred because:

1. They are easier to back up or migrate.
2. They can be managed using Docker CLI commands.
3. They work across all Docker hosts and operating systems.
4. New volumes can have their content pre-populated by a container.
5. They are more performant for I/O operations on some file systems.

AI Feedback:

No answer provided.

Q25. [Docker] ' INCORRECT – 0/5

Question:

Explain how Docker networking works. How can containers communicate with each other and with the outside world?

Your Answer:

Not answered

Ideal Answer:

Docker provides several networking drivers (e.g., bridge, host, null, overlay, macvlan). The default is `bridge` network. When containers are attached to the same bridge network, they can communicate with each other using their container names or IP addresses. To communicate with the outside world (host machine or external network), specific ports must be mapped from the container to the host using the `-p` or `--publish` flag (e.g., `-p 8080:80`). This makes the container's internal port accessible via the host's IP and the specified host port. Containers can also communicate with external services if the host machine has network access.

AI Feedback:

No answer provided.

Q26. [Docker] ' INCORRECT – 0/5

Question:

What is Docker Compose and when would you use it? Provide a simple example scenario where Docker Compose would be beneficial.

Your Answer:

Not answered

Ideal Answer:

Docker Compose is a tool for defining and running multi-container Docker applications. It allows you to define all the services (containers), networks, and volumes for your application in a single YAML file (`docker-compose.yml`) and then start, stop, and manage them all with a single command (`docker-compose up`). You would use it for development and testing environments where you need to orchestrate multiple interconnected services, such as a web application with a separate database and a caching service. For example, a web app (Node.js/Python), a PostgreSQL database, and a Redis cache can all be defined and managed together with Docker Compose.

AI Feedback:

No answer provided.

Q27. [Docker] ' INCORRECT – 0/5

Question:

You have a Docker container that is constantly crashing. Describe the steps you would take to troubleshoot and diagnose the issue.

Your Answer:

Not answered

Ideal Answer:

To troubleshoot a crashing Docker container, I would follow these steps:

1. **Check Logs:** Use `docker logs <container_id_or_name>` to view the application logs. The error messages here are often the most direct indication of the problem.
2. **Inspect Container:** Use `docker inspect <container_id_or_name>` to check details about its configuration, network settings, exit code, and state.
3. **Run in Interactive Mode:** Try running the container with `docker run -it --rm <image_name> /bin/bash` (or appropriate shell) to get an interactive shell, then manually execute the entrypoint command or application to see errors in real-time.
4. **Resource Limits:** Check if the container is running out of memory or CPU. Docker Desktop or `docker stats` can provide real-time resource usage.
5. **Dockerfile Review:** Examine the Dockerfile for incorrect commands, missing dependencies, or misconfigurations.
6. **Image History:** Check `docker history <image_name>` to understand how the image was built layer by layer.

AI Feedback:

No answer provided.

Q28. [Docker] ' INCORRECT – 0/5

Question:

Explain the concept of 'multi-stage builds' in a Dockerfile and why they are beneficial.

Your Answer:

Not answered

Ideal Answer:

Multi-stage builds allow you to use multiple `FROM` statements in your Dockerfile, each new `FROM` instruction starting a new build stage. You can selectively copy artifacts from one stage to another, leaving behind everything you don't need from the previous stage. This is beneficial because:

1. **Reduced Image Size:** It significantly reduces the final image size by excluding build tools, SDKs, and intermediate files that are only needed during the build process, not at runtime.
2. **Improved Security:** Smaller images have a smaller attack surface.

3. **Cleaner Images:** Separates build dependencies from runtime dependencies, resulting in a cleaner and more efficient final image.

AI Feedback:

No answer provided.

Q29. [Docker] ' INCORRECT – 0/5

Question:

How would you secure a Docker container and its environment? List at least three best practices for Docker security.

Your Answer:

Not answered

Ideal Answer:

Securing Docker involves practices for images, containers, and the host system. Best practices include:

1. **Use Official and Trusted Base Images:** Start with minimal, official, and regularly updated base images (e.g., Alpine Linux).
2. **Minimize Image Size:** Use multi-stage builds and remove unnecessary packages, libraries, and build tools to reduce the attack surface.
3. **Run Containers as Non-Root Users:** Avoid running processes inside the container as the root user. Use the `USER` instruction in the Dockerfile.
4. **Implement Least Privilege:** Only expose necessary ports (`EXPOSE` vs. `-p`), limit resource usage, and avoid granting unnecessary capabilities or access to host resources.
5. **Scan Images for Vulnerabilities:** Integrate tools like Clair or Docker Scout into your CI/CD pipeline to scan for known vulnerabilities.
6. **Don't Store Sensitive Data in Images:** Use Docker secrets or environment variables for credentials and sensitive information, not hardcoded in the Dockerfile or image.

AI Feedback:

No answer provided.

Q30. [Docker] ' INCORRECT – 0/5

Question:

Explain the difference between `CMD` and `ENTRYPOINT` instructions in a Dockerfile. When would you use one over the other or combine them?

Your Answer:

Not answered

Ideal Answer:

Both `CMD` and `ENTRYPOINT` define what command gets executed when a container starts from an image.

- **`CMD`:** Provides defaults for an executing container. These defaults can include an executable, or they can omit the executable, in which case you must specify an `ENTRYPOINT` instruction. If `CMD` is used alone, it will be executed directly. If `CMD` is used with `ENTRYPOINT`, `CMD`'s arguments are appended to the `ENTRYPOINT` instruction's arguments. Only the last `CMD` instruction in a Dockerfile takes effect. It is easily overridden when running `docker run`.
- **`ENTRYPOINT`:** Configures a container that will run as an executable. It defines the command that will always be executed when the container starts. Any `CMD` instructions or command-line arguments to `docker run` are appended as arguments to the `ENTRYPOINT`. It's typically used when building an image for a specific application that you want to run as the main process (e.g., `ENTRYPOINT ['nginx']`).

Usage:

- Use `CMD` alone for a container that you expect users to override often (e.g., a base OS image where users might run different commands).
- Use `ENTRYPOINT` alone when you want the container to behave like an executable (e.g., an image for a specific application like `apache`, `nginx`).
- Combine `ENTRYPOINT` and `CMD` when you want to define a fixed executable and provide default arguments that can be easily overridden (e.g., `ENTRYPOINT ['/app/my_app']`, `CMD ['--config', '/etc/my_app.conf']`).

AI Feedback:

No answer provided.

Q31. [DSA] ' INCORRECT – 0/5

Question:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution, and you may not use the same element twice.

Your Answer:

Not answered

Ideal Answer:

A brute-force approach uses nested loops ($O(n^2)$). A more efficient approach uses a hash map. Iterate through the array; for each number `num`, calculate `complement = target - num`. Check if `complement` exists in the hash map. If it does, return the current index and the index of the complement. Otherwise, add `num` and its index to the hash map. This approach is $O(n)$ time and $O(n)$ space.

AI Feedback:

No answer provided.

Q32. [DSA] ' INCORRECT – 0/5

Question:

Reverse a singly linked list.

Your Answer:

Not answered

Ideal Answer:

Iteratively, maintain three pointers: `prev` (null), `current` (head), and `next\_temp`. In a loop, store `current.next` in `next\_temp`, then set `current.next` to `prev`. Move `prev` to `current`, and `current` to `next\_temp`. Repeat until `current` is null. `prev` will be the new head. Recursively, if the list is empty or has one node, return the head. Otherwise, reverse the rest of the list (`newHead = reverseList(head.next)`), then make `head.next.next` point to `head`, and set `head.next` to null. Return `newHead`.

AI Feedback:

No answer provided.

Q33. [DSA] ' INCORRECT – 0/5

Question:

Given the `root` of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Your Answer:

Not answered

Ideal Answer:

Use a Breadth-First Search (BFS) approach. Initialize a queue and add the root node. While the queue is not empty, get the size of the current level. Create a list to store nodes' values for this level. Iterate `size` times: dequeue a node, add its value to the current level list, and enqueue its left and right children if they exist. After the loop, add the current level list to the overall result list.

AI Feedback:

No answer provided.

Q34. [DSA] ' INCORRECT – 0/5

Question:

Given an $m \times n$ 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

Your Answer:

Not answered

Ideal Answer:

Use Depth-First Search (DFS) or Breadth-First Search (BFS). Iterate through each cell of the grid. If a cell contains '1', increment the island count and start a traversal (DFS or BFS) from that cell to mark all connected '1's as visited (e.g., change them to '0' or a different character) to avoid recounting. DFS involves a recursive function that explores all four adjacent cells. BFS uses a queue.

AI Feedback:

No answer provided.

Q35. [DSA] ' INCORRECT – 0/5

Question:

Find the k -th largest element in an unsorted array. Note that it is the k -th largest element in the sorted order, not the k -th distinct element.

Your Answer:

Not answered

Ideal Answer:

A common approach is using a min-heap (priority queue). Iterate through the array, adding each element to the min-heap. If the heap's size exceeds k , remove the smallest element (`heap.poll()`). After processing all elements, the root of the min-heap (`heap.peek()`) will be the k -th largest element. This takes $O(N \log k)$ time. Alternatively, Quickselect can find the k -th largest in average $O(N)$ time.

AI Feedback:

No answer provided.

Q36. [DSA] ' INCORRECT – 0/5

Question:

Given a string s , find the first non-repeating character in it and return its index. If it does not exist, return -1.

Your Answer:

Not answered

Ideal Answer:

Use a hash map (or frequency array for ASCII characters) to store the frequency of each character. Iterate through the string once to populate the frequency map. Then, iterate through the string a second time. For each character, check its frequency in the map. The first character encountered with a frequency of 1 is the answer. Return its index. If no such character is found, return -1. Time complexity is $O(N)$ and space complexity is $O(1)$ for fixed alphabet size.

AI Feedback:

No answer provided.

Q37. [DSA] ' INCORRECT – 0/5

Question:

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Your Answer:

Not answered

Ideal Answer:

This is a classic dynamic programming problem. Let $dp[i]$ be the number of distinct ways to reach step i . To reach step i , you could have come from step $i-1$ (by taking 1 step) or from step $i-2$ (by taking 2 steps). So, $dp[i] = dp[i-1] + dp[i-2]$. The base cases are $dp[1] = 1$ and $dp[2] = 2$. This is the Fibonacci sequence. The time complexity is $O(N)$ and space complexity is $O(N)$ (or $O(1)$ if optimized).

AI Feedback:

No answer provided.

Q38. [DSA] ' INCORRECT – 0/5

Question:

Given a collection of distinct integers, return all possible permutations. You can return the answer in any order.

Your Answer:

Not answered

Ideal Answer:

This can be solved using backtracking. Define a recursive function that takes the current permutation and a list of remaining numbers. In each step, iterate through the remaining numbers. For each number, add it to the current permutation, remove it from the remaining numbers, and recursively call the function. After the recursive call returns, backtrack by removing the number from the current permutation and adding it back to the remaining numbers. The base case is when there are no remaining numbers; add the current permutation to the result list.

AI Feedback:

No answer provided.

Q39. [DSA] ' INCORRECT – 0/5

Question:

Implement binary search for a sorted array. Return the index of the target if found, otherwise -1.

Your Answer:

Not answered

Ideal Answer:

Binary search works on a sorted array by repeatedly dividing the search interval in half. Initialize `left` to 0 and `right` to `array.length - 1`. While `left <= right`, calculate `mid = left + (right - left) / 2`. If `array[mid]` equals the `target`, return `mid`. If `array[mid]` is less than `target`, set `left = mid + 1` (search in the right half). If `array[mid]` is greater than `target`, set `right = mid - 1` (search in the left half). If the loop finishes, return -1. Time complexity is $O(\log N)$.

AI Feedback:

No answer provided.

Q40. [DSA] ' INCORRECT – 0/5

Question:

Design and implement a data structure for a Least Recently Used (LRU) cache. It should support `get` and `put` operations in $O(1)$ time.

Your Answer:

Not answered

Ideal Answer:

An LRU cache can be implemented using a combination of a hash map (`HashMap<Key, Node>`) and a doubly linked list. The hash map stores `key -> Node` mappings. The doubly linked list maintains the order of usage, with the most recently used at one end (e.g., tail) and the least recently used at the other end (e.g., head). `get(key)` checks the map, moves the node to MRU end, and returns value. `put(key, value)` updates or inserts; if inserting and capacity is reached, removes LRU node from head, then adds new node to tail. Both operations are $O(1)$ on average.

AI Feedback:

No answer provided.

Q41. [DBMS] ' INCORRECT – 0/5

Question:

Explain database normalization. What are the first three normal forms, and why is normalization important?

Your Answer:

Not answered

Ideal Answer:

Database normalization is the process of organizing the columns and tables of a relational database to minimize data redundancy and improve data integrity.

- **1NF (First Normal Form)**: Each column contains atomic values, and there are no repeating groups of columns.

- **2NF (Second Normal Form)**: Is in 1NF and all non-key attributes are fully functionally dependent on the primary key (no partial dependencies).

- **3NF (Third Normal Form)**: Is in 2NF and all non-key attributes are not transitively dependent on the primary key (no transitive dependencies).

Importance: Reduces data redundancy, improves data integrity, simplifies data maintenance, and makes the database schema more flexible and easier to query.

AI Feedback:

No answer provided.

Q42. [DBMS] ' INCORRECT – 0/5

Question:

What are the ACID properties in the context of database transactions? Explain each.

Your Answer:

Not answered

Ideal Answer:

ACID is an acronym representing a set of properties that guarantee valid transactions:

- **Atomicity**: A transaction is treated as a single, indivisible unit of work. Either all operations within it are completed, or none are.

- **Consistency**: A transaction brings the database from one valid state to another. It ensures that data conforms to all defined rules and constraints.

- **Isolation**: Concurrent transactions execute in isolation from each other. The intermediate state of a transaction is not visible to other transactions until it is committed.

- **Durability**: Once a transaction has been committed, it will remain committed even in the event of system failures (e.g., power loss, crashes).

AI Feedback:

No answer provided.

Q43. [DBMS] ' INCORRECT – 0/5

Question:

How do indexes work in a database, and what are the trade-offs of using them?

Your Answer:

Not answered

Ideal Answer:

An index is a data structure (like a B-tree or hash map) that improves the speed of data retrieval operations on a database table at the cost of additional writes and storage space. It provides a quick lookup mechanism for rows based on the values in one or more columns.

- **How it works**: When a query searches for data, the database can use the index to quickly locate the relevant rows without scanning the entire table.

- **Trade-offs**:

- **Pros**: Significantly speeds up SELECT queries, especially for large tables; can enforce uniqueness.

- **Cons**: Slows down INSERT, UPDATE, and DELETE operations because the index must also be updated; consumes additional disk space; can sometimes be misused, leading to slower queries.

AI Feedback:

No answer provided.

Q44. [DBMS] ' INCORRECT – 0/5

Question:

Compare and contrast SQL and NoSQL databases, highlighting their primary use cases.

Your Answer:

Not answered

Ideal Answer:

****SQL (Relational Databases)**:**

- ****Structure**:** Tabular, predefined schema (rows and columns).
- ****ACID**:** Strong ACID compliance for transactions.
- ****Scalability**:** Primarily vertical scaling; horizontal scaling (sharding) is possible but more complex.
- ****Query Language**:** SQL.
- ****Use Cases**:** Financial transactions, systems requiring high data integrity, complex queries, traditional enterprise applications.

****NoSQL (Non-relational Databases)**:**

- ****Structure**:** Document (JSON), Key-Value, Column-Family, Graph. Schema-less or flexible schema.
- ****ACID**:** Often sacrifices some ACID properties for availability and partition tolerance (BASE properties).
- ****Scalability**:** Designed for horizontal scaling.
- ****Query Language**:** Varies by database (e.g., query APIs).
- ****Use Cases**:** Big data, real-time web apps, mobile apps, content management, social networks, IoT.

AI Feedback:

No answer provided.

Q45. [DBMS] ' INCORRECT – 0/5

Question:

Explain database transaction isolation levels and their importance in managing concurrency.

Your Answer:

Not answered

Ideal Answer:

Isolation levels determine how and when changes made by one transaction become visible to other concurrent transactions. They balance data integrity against concurrency performance.

- ****Read Uncommitted**:** Dirty reads, non-repeatable reads, phantom reads are possible. Lowest isolation, highest concurrency.
- ****Read Committed**:** Prevents dirty reads. Non-repeatable reads and phantom reads are possible.
- ****Repeatable Read**:** Prevents dirty reads and non-repeatable reads. Phantom reads are possible.
- ****Serializable**:** Highest isolation level. Prevents dirty reads, non-repeatable reads, and phantom reads. Achieves full isolation at the cost of reduced concurrency.

Importance: Choosing the right isolation level is crucial for preventing concurrency issues (like lost updates, inconsistent reads) while ensuring an acceptable level of performance for multi-user database systems.

AI Feedback:

No answer provided.

Q46. [DBMS] ' INCORRECT – 0/5

Question:

Describe the different types of SQL JOINS and provide an example of when you would use each.

Your Answer:

Not answered

Ideal Answer:

SQL JOINS combine rows from two or more tables based on a related column between them:

- ****INNER JOIN**:** Returns rows when there is a match in both tables. Example: Get orders with customer details only if the customer exists.
- ****LEFT (OUTER) JOIN**:** Returns all rows from the left table, and the matching rows from the right table. If no match, NULLs for right table's columns. Example: Get all customers and their orders, even if a customer has no orders.

- ****RIGHT (OUTER) JOIN****: Returns all rows from the right table, and the matching rows from the left table. If no match, NULLs for left table's columns. Example: Get all products and details of orders containing them, even if a product has never been ordered.
- ****FULL (OUTER) JOIN****: Returns all rows when there is a match in one of the tables. If no match, NULLs for the non-matching table's columns. Example: Get all customers and all products, showing which customers bought which products, even if a customer bought nothing or a product was never bought.

AI Feedback:

No answer provided.

Q47. [DBMS] ' INCORRECT – 0/5

Question:

What are Stored Procedures, Triggers, and Views in a database, and how do they differ in their purpose?

Your Answer:

Not answered

Ideal Answer:

- **Stored Procedure**: A pre-compiled collection of SQL statements stored in the database. It can accept parameters, return values, and execute complex logic. Purpose: Encapsulate business logic, improve performance, enhance security, reduce network traffic.
- **Trigger**: A special type of stored procedure that automatically executes (fires) in response to certain events on a table (INSERT, UPDATE, DELETE). Purpose: Enforce complex business rules, maintain data integrity, audit changes, propagate changes to other tables.
- **View**: A virtual table based on the result-set of a SQL query. It does not store data itself but rather a definition of how to retrieve the data from one or more underlying tables. Purpose: Simplify complex queries, enhance security (restrict access to certain columns/rows), provide data abstraction.

AI Feedback:

No answer provided.

Q48. [DBMS] ' INCORRECT – 0/5

Question:

Explain database sharding and partitioning. When would you use each, and what are their benefits?

Your Answer:

Not answered

Ideal Answer:

- **Partitioning**: Dividing a large table into smaller, more manageable pieces (partitions) within the *same database instance*. Data is distributed based on a rule (e.g., range, list, hash) across partitions.
 - Use Cases: Improve performance for large tables, simplify data management, reduce index size.
 - Benefits: Query performance, manageability, maintenance.
 - **Sharding**: Distributing data from a single logical database across multiple *separate database instances* (shards), potentially on different servers. Each shard holds a subset of the total data.
 - Use Cases: Handle massive datasets and high transaction volumes, overcome single-server limitations, achieve horizontal scalability.
 - Benefits: Extreme scalability, high availability, improved write performance.
- Difference: Partitioning keeps data on one database instance, while sharding distributes it across multiple instances. Sharding is more complex but offers greater scalability.

AI Feedback:

No answer provided.

Q49. [DBMS] ' INCORRECT – 0/5

Question:

Explain the CAP theorem in the context of distributed databases.

Your Answer:

Not answered

Ideal Answer:

The CAP theorem states that a distributed data store can only simultaneously guarantee two of the following three properties:

- **Consistency (C)**: All nodes see the same data at the same time. A read always returns the most recent write.
- **Availability (A)**: Every request receives a response about whether it was successful or failed – without guaranteeing that the response reflects the most recent write.
- **Partition Tolerance (P)**: The system continues to operate even if there are network failures (partitions) that prevent some nodes from communicating with others.

Explanation: In a distributed system, partitions are inevitable. Therefore, you must choose between

Consistency and Availability during a network partition. CP systems prioritize consistency, while AP systems prioritize availability.

AI Feedback:

No answer provided.

Q50. [DBMS] ' INCORRECT – 0/5

Question:

How would you go about optimizing a slow SQL query? What tools or techniques would you use?

Your Answer:

Not answered

Ideal Answer:

Optimizing a slow SQL query involves identifying bottlenecks and making changes to improve its execution time.

Techniques and Tools:

1. **Use `EXPLAIN` or `EXPLAIN ANALYZE` (or similar for other DBs):** To understand the query execution plan and identify slow parts (e.g., full table scans, expensive joins).
2. **Indexing:** Ensure appropriate indexes exist on columns used in `WHERE` clauses, `JOIN` conditions, `ORDER BY`, and `GROUP BY`. Avoid over-indexing.
3. **Rewrite Query:** Select only necessary columns (avoid `SELECT \*`), optimize `WHERE` clauses (avoid functions on indexed columns), improve `JOIN` efficiency, minimize `ORDER BY`/`GROUP BY` operations.
4. **Denormalization:** In specific cases, intentionally adding redundancy can reduce complex joins and improve read performance.
5. **Analyze Database Statistics:** Ensure the database's query optimizer has up-to-date statistics on table and index data distributions.
6. **Hardware/Configuration:** Review server resources (RAM, CPU, I/O) and database configuration parameters.

AI Feedback:

No answer provided.

Q51. [DAA] ' INCORRECT – 0/5

Question:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. Assume that each input would have exactly one solution, and you may not use the same element twice.

Your Answer:

Not answered

Ideal Answer:

A common approach is to use a hash map (or dictionary). Iterate through the array. For each number `num`, calculate `complement = target - num`. Check if `complement` exists in the hash map. If it does, you've found the pair and return their indices. If not, add `num` and its index to the hash map. This approach has a time complexity of $O(n)$ and space complexity of $O(n)$.

AI Feedback:

No answer provided.

Q52. [DAA] ' INCORRECT – 0/5

Question:

Reverse a singly linked list.

Your Answer:

Not answered

Ideal Answer:

You can reverse a linked list iteratively or recursively. The iterative approach involves using three pointers: `prev`, `current`, and `next\_temp`. Initialize `prev` to `null` and `current` to the `head`. In each iteration, store `current.next` in `next\_temp`, then set `current.next` to `prev`, update `prev` to `current`, and finally update `current` to `next\_temp`. Continue until `current` is `null`. `prev` will then be the new head. Time complexity is $O(n)$, space complexity is $O(1)$.

AI Feedback:

No answer provided.

Q53. [DAA] ' INCORRECT – 0/5

Question:

Given the `root` of a binary tree, determine if it is a valid binary search tree (BST). A valid BST is defined as follows: The left subtree of a node contains only nodes with keys less than the node's key. The right subtree of a node contains only nodes with keys greater than the node's key. Both the left and right subtrees must also be binary search trees.

Your Answer:

Not answered

Ideal Answer:

The most robust approach is to perform an in-order traversal while keeping track of the `min` and `max` allowed values for each node. For the root, `min` can be $-\infty$ and `max` can be $+\infty$. When traversing to the left child, update the `max` bound to the parent's value. When traversing to the right child, update the `min` bound to the parent's value. If any node's value violates its `min` or `max` bound, it's not a valid BST. Alternatively, an in-order traversal of a BST produces a sorted sequence. You can collect the elements in an array during in-order traversal and then check if the array is sorted.

AI Feedback:

No answer provided.

Q54. [DAA] ' INCORRECT – 0/5

Question:

Given a 2D grid representing a maze with '0's (open paths) and '1's (walls), and a start and end point, find the shortest path from start to end. If no path exists, return -1. Assume movement is restricted to 4 directions (up, down, left, right).

Your Answer:

Not answered

Ideal Answer:

This problem can be solved using Breadth-First Search (BFS). BFS naturally finds the shortest path in an unweighted graph. Start by putting the `start` coordinate into a queue and mark it as visited. In each step, dequeue a cell, and if it's the `end` cell, return the distance. Otherwise, explore its valid (within bounds, not a wall, not visited) neighbors. For each valid neighbor, enqueue it, mark it visited, and increment its distance from the start. If the queue becomes empty and the `end` is not reached, no path exists.

AI Feedback:

No answer provided.

Q55. [DAA] ' INCORRECT – 0/5

Question:

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0. A subsequence is a sequence that can be derived from another sequence by deleting some or no characters without changing the order of the remaining characters.

Your Answer:

Not answered

Ideal Answer:

This is a classic dynamic programming problem. Create a 2D DP table `dp[m+1][n+1]`, where `m` and `n` are the lengths of `text1` and `text2`. `dp[i][j]` will store the length of the LCS of `text1[0...i-1]` and `text2[0...j-1]`. Initialize the first row and column to 0. Iterate through the table: if `text1[i-1] == text2[j-1]`, then `dp[i][j] = 1 + dp[i-1][j-1]`. Otherwise, `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`. The final answer is `dp[m][n]`. Time complexity is $O(m*n)$, space complexity is $O(m*n)$.

AI Feedback:

No answer provided.

Q56. [DAA] ' INCORRECT – 0/5

Question:

Explain the Merge Sort algorithm and analyze its time and space complexity.

Your Answer:

Not answered

Ideal Answer:

Merge Sort is a divide-and-conquer sorting algorithm. It works by recursively dividing the array into two halves until individual elements are reached. Then, it merges these sorted halves back together to produce a sorted array. The `merge` step is crucial: it takes two sorted subarrays and combines them into one sorted array by comparing elements from both halves and placing the smaller one into the result array. Time complexity is $O(n \log n)$ in all cases (best, average, worst) due to the balanced division and linear merge time. Space complexity is $O(n)$ because a temporary array of size n is needed for merging.

AI Feedback:

No answer provided.

Q57. [DAA] ' INCORRECT – 0/5

Question:

Implement a first-in-first-out (FIFO) queue using only two stacks. The implemented queue should support `push`, `pop`, `peek`, and `empty` operations.

Your Answer:

Not answered

Ideal Answer:

Use two stacks, `s1` (for pushing) and `s2` (for popping/peeking).

- `push(x)` : Simply push `x` onto `s1`.

- `pop()` / `peek()` : Before popping or peeking, check if `s2` is empty. If it is, transfer all elements from `s1` to `s2` (this reverses their order). After transferring, `s2` will have the elements in FIFO order. Then, perform the pop/peek operation on `s2`. If `s2` is still empty after transfer (meaning `s1` was also empty), the queue is empty.

- `empty()` : Return true if both `s1` and `s2` are empty.

The amortized time complexity for `push`, `pop`, `peek` is $O(1)$ because each element is pushed onto `s1` once and moved to `s2` once.

AI Feedback:

No answer provided.

Q58. [DAA] ' INCORRECT – 0/5

Question:

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array. Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Your Answer:

Not answered

Ideal Answer:

A common approach is to use a min-heap (priority queue) of size `k`. Iterate through the array `nums`. For each number, add it to the min-heap. If the heap's size exceeds `k`, remove the smallest element (the root) from the heap. After iterating through all elements, the root of the min-heap will be the `k`th largest element. This approach has a time complexity of $O(N \log k)$ because N elements are processed, and each heap operation (insert/delete) takes $O(\log k)$ time. Space complexity is $O(k)$. Another approach is using QuickSelect (average $O(N)$).

AI Feedback:

No answer provided.

Q59. [DAA] ' INCORRECT – 0/5

Question:

Explain Dijkstra's algorithm for finding the shortest paths from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.

Your Answer:

Not answered

Ideal Answer:

Dijkstra's algorithm is a greedy algorithm that finds the shortest path from a source node to all other nodes in a graph with non-negative edge weights. It maintains a set of visited nodes and a distance array `dist[]` initialized to infinity for all nodes except the source (which is 0). It uses a min-priority queue to efficiently select the unvisited node with the smallest current distance. In each step, it extracts the node `u` with the minimum distance from the priority queue, marks it as visited, and then relaxes all its unvisited neighbors `v`. Relaxing means updating `dist[v]` if `dist[u] + weight(u,v)` is less than `dist[v]`. The process continues until the priority queue is empty or all reachable nodes are visited. Time complexity is typically $O(E \log V)$ or $O(E + V \log V)$ with a Fibonacci heap, and $O(V^2)$ with an adjacency matrix.

AI Feedback:

No answer provided.

Q60. [DAA] ' INCORRECT – 0/5

Question:

Given an array `nums` of distinct integers, return all possible permutations. You may return the answer in any order.

Your Answer:

Not answered

Ideal Answer:

This can be solved using backtracking. Define a recursive function `backtrack(current_permutation, remaining_elements)`.

- **Base Case:** If `remaining_elements` is empty, then `current_permutation` is a complete permutation, so add it to the results.

- **Recursive Step:** Iterate through `remaining_elements`. For each `num` in `remaining_elements`:

- Add `num` to `current_permutation`.

- Recursively call `backtrack` with the updated `current_permutation` and `remaining_elements` excluding `num`.

- **Backtrack:** Remove `num` from `current_permutation` to explore other possibilities.

The initial call would be `backtrack([], nums)`. The time complexity is $O(n * n!)$ because there are $n!$ permutations, and generating each one takes $O(n)$ time to copy/build. Space complexity is $O(n)$ for the recursion stack and temporary storage for permutations.

AI Feedback:

No answer provided.

Q61. [HR] ' INCORRECT – 0/5

Question:

Describe a situation where you had a strong disagreement with a team member or a superior about a technical approach or solution. How did you handle it?

Your Answer:

Not answered

Ideal Answer:

A good answer details the technical disagreement, demonstrates active listening to the other's perspective, articulates their own reasoning with data or evidence, and shows an ability to compromise or align with the team's final decision for the project's success. It emphasizes professional communication and focuses on the best outcome for the project rather than personal victory.

AI Feedback:

No answer provided.

Q62. [HR] ' INCORRECT – 0/5

Question:

Tell me about a significant technical project or feature where you faced a major setback or failure. What happened, how did you react, and what did you learn from it?

Your Answer:

Not answered

Ideal Answer:

The candidate should clearly explain the technical challenge or failure, take responsibility if applicable, describe their problem-solving and debugging process, and outline the corrective actions taken. Crucially, they must articulate specific lessons learned (e.g., improved testing, better communication, different architectural considerations) and how they applied these learnings to subsequent work.

AI Feedback:

No answer provided.

Q63. [HR] ' INCORRECT – 0/5

Question:

Imagine you're working on a critical bug fix for a production system, and simultaneously, your manager assigns you a high-priority new feature request. How would you prioritize your tasks and communicate your plan?

Your Answer:

Not answered

Ideal Answer:

A strong answer involves evaluating the immediate impact and urgency of both tasks, consulting with the manager or relevant stakeholders for clarification on priorities, and proposing a reasoned plan. This might include time-boxing, partial completion, delegation (if possible), or escalating to get more resources, always coupled with clear communication about expected timelines and potential trade-offs.

AI Feedback:

No answer provided.

Q64. [HR] ' INCORRECT – 0/5

Question:

Describe a time you had to quickly learn a new programming language, framework, or technical skill for a project. What was your approach to learning, and how did you apply it?

Your Answer:

Not answered

Ideal Answer:

The candidate should describe the specific technology and the project context. They should detail their proactive learning process (e.g., reading documentation, online courses, personal projects, seeking mentorship), discuss any challenges faced, and demonstrate how they successfully integrated the new skill to contribute effectively to the project.

AI Feedback:

No answer provided.

Q65. [HR] ' INCORRECT – 0/5

Question:

Tell me about a project where the technical requirements were vague or changed frequently. How did you handle this ambiguity and ensure a successful outcome?

Your Answer:

Not answered

Ideal Answer:

A good response highlights proactive communication skills: asking clarifying questions, scheduling regular check-ins with stakeholders, documenting assumptions, and iteratively refining solutions. It demonstrates adaptability, a structured approach to breaking down complex problems, and the ability to manage expectations while navigating evolving requirements.

AI Feedback:

No answer provided.

Q66. [HR] ' INCORRECT – 0/5

Question:

Recount a time when you received constructive criticism on your technical work (e.g., code review, design document). How did you react and what did you do with that feedback?

Your Answer:

Not answered

Ideal Answer:

The candidate should show receptiveness to feedback, even if initially challenging. They should describe how they processed the criticism objectively, asked clarifying questions, and took concrete steps to incorporate the feedback (e.g., refactoring code, revising a design, seeking further learning). It demonstrates humility, a growth mindset, and a commitment to quality.

AI Feedback:

No answer provided.

Q67. [HR] ' INCORRECT – 0/5

Question:

Describe a situation where you took initiative to mentor a less experienced technical colleague or led a small technical initiative within your team. What was the impact?

Your Answer:

Not answered

Ideal Answer:

A strong answer outlines the specific scenario, the actions taken to mentor or lead, challenges encountered, and how they were overcome. The candidate should clearly articulate the positive impact on the individual (e.g., improved skills, confidence), the team (e.g., increased productivity, knowledge sharing), or the project (e.g., successful feature delivery, process improvement).

AI Feedback:

No answer provided.

Q68. [HR] ' INCORRECT – 0/5

Question:

Tell me about a challenging situation where you had to interact with a non-technical stakeholder or client regarding a complex technical issue. How did you manage their expectations and ensure understanding?

Your Answer:

Not answered

Ideal Answer:

The candidate should demonstrate strong communication skills by explaining how they translated complex technical jargon into easily understandable terms for a non-technical audience. They should highlight active listening, empathy, managing expectations regarding timelines or limitations, and collaboratively finding solutions that address both technical feasibility and business needs.

AI Feedback:

No answer provided.

Q69. [HR] ' INCORRECT – 0/5

Question:

Describe a situation where you encountered a potential ethical dilemma or a conflict of interest in a technical project or decision. How did you approach it?

Your Answer:

Not answered

Ideal Answer:

A good response demonstrates a strong ethical compass. The candidate should describe the specific dilemma, how they recognized its ethical implications, and the steps they took to address it (e.g., consulting company policy, speaking with a manager or HR, prioritizing data privacy or security). It highlights integrity and professional responsibility in their decision-making process.

AI Feedback:

No answer provided.

Q70. [HR] ' INCORRECT – 0/5

Question:

Beyond your assigned project tasks, what technical contributions have you made to your team or organization (e.g., improving processes, tooling, documentation, knowledge sharing)?

Your Answer:

Not answered

Ideal Answer:

The candidate should provide concrete examples of initiatives they took outside their core responsibilities. This could include automating tedious tasks, creating better internal documentation, developing a useful tool, or organizing knowledge-sharing sessions. They should clearly articulate the problem solved, the actions taken, and the positive impact on the team's efficiency, quality, or collaboration.

AI Feedback:

No answer provided.
